

LAB 02 • CLASS 1→2 • DIRECTING

The Daily Briefing Agent

Teach Ghost what actually matters to you, then make it publish a living briefing page it re-generates on command — cutting the noise until only the signal survives.

DIFFICULTY EASY+

TIME ~60-75 MIN

REINFORCES CLASS 1→2 • DIRECTING

YOU LEAVE WITH A LIVE, RE-GENERATING BRIEFING PAGE

LEARNING OBJECTIVES

- Prime Ghost with durable **priorities** — your topics, projects, and trusted sources — plus a strict output **format**, so future runs inherit your standards without re-explaining.
- Direct Ghost to build a briefing from the **live web** filtered through what it already knows you care about, and deploy it as a **bookmarkable page**.
- Debug **relevance, not prose** — raise the signal threshold so the briefing stops over-reporting and starts earning its place on your screen.
- Crystallize the working behavior into a saved **daily-briefing workflow** with the relevance rules baked in.
- Understand why the same lesson holds whether Ghost is **student-triggered or self-waking**.

Why this lab exists

Anyone can ask an AI for "today's news." That is a search, not an agent. The difference — and the entire point of Class 2 — is **direction**: an agent that knows your projects well enough to decide what is worth telling you, and what to leave out.

The hard skill here is not writing. Ghost writes clean bullets on its first try. The hard skill is *relevance*. A briefing that reports everything is just a firehose with better grammar; you will stop reading it by Thursday. In this lab you will build the firehose on purpose, watch it fail as a daily habit, and then **steer Ghost's judgment** until the page shows only what changes a decision you actually have to make. That act — tuning what an agent chooses to surface — is the core of directing, and it transfers to every agent you build after this one.

You also produce a **persistent artifact**: not a chat reply that scrolls away, but a URL you can bookmark, where today's briefing quietly replaces yesterday's each time Ghost runs.

Prerequisites

- **Lab 0 and Lab 1 complete.** Ghost is set up and you have deployed at least one live page, so the "deploy a real URL" muscle already exists.
- **Ghost is live in your Discord.** All interaction in this lab happens in your Discord channel — that is where you talk to Ghost.

- **Memory is working.** Confirm Ghost remembers something across sessions (ask it what it knows about you). Priming is worthless if it forgets by tomorrow.
- Have **2–4 real projects or interests** in mind before you start. Vague inputs ("tech, sports") produce vague briefings. Specific stakes ("shipping our billing rewrite in Q3") are what make relevance filtering possible.

The build — step by step

1

Prime — save your priorities and the format

Before asking for anything, teach Ghost what you care about and exactly how you want it delivered. Be concrete about projects, because projects are what the relevance filter will key off later. Lock the format now so every future briefing looks the same.

```
# SAVE THIS TO MEMORY — MY BRIEFING PRIORITIES
```

```
Remember these as my standing briefing priorities.
```

```
Projects I'm actively working on (rank in this order):
```

1. Billing rewrite — migrating our payments to a new provider, shipping Q3
2. Hiring a senior backend engineer
3. My side project: a local-first note-taking app

```
Topics I track: LLM agent tooling, payments/fintech regulation, remote hiring trends.
```

```
Trusted sources: prefer primary sources, official blogs, and reputable trade press over aggregators.
```

```
FORMAT for every briefing (do not deviate):
```

- Fits on one screen. No preamble, no sign-off, no filler.
- Group by section. Each item = a bold headline + at most 3 tight bullets.
- Plain language. If it doesn't change something I'd do, it doesn't belong.

2

Prompt — build and deploy today's briefing

Now direct Ghost to compile the briefing from the web, filtered through what it just saved, and publish it as a live page you can bookmark. Ask it to treat the page as *the* briefing — regenerating in place rather than spawning a new URL each time.

```
# BUILD + DEPLOY THE BRIEFING PAGE
```

Using what you know about my priorities, build today's briefing from the current web.

Then deploy it as a single live page I can bookmark. When I later ask for "today's briefing," regenerate that same page in place — today's replaces yesterday's, same URL. Put today's date at the top.

Give me the link.

You now have a URL. Open it. It will probably look impressive — and that is the trap.

3

Break & steer — debug the relevance, not the writing

Read the first version as if it landed in your inbox every morning. You will notice it **over-reports**: general industry news, mildly-interesting items with no bearing on your projects, three payments stories when one moved the needle. The prose is fine. The *judgment* is not. This is the lesson — steer the threshold, not the style.

```
# RAISE THE SIGNAL THRESHOLD
```

This is too noisy — you're reporting things that don't affect me.

Tighten the rules and rebuild the same page:

- Only include an item if it actually affects one of my ranked projects. If it's just "interesting," cut it.
- If nothing qualifies for a section, DROP the section entirely. Don't pad it to look complete.
- Rank items by how much they matter to my projects, not by how recent they are. Project #1 outranks a fresher story about project #3.
- When in doubt, leave it out. I'd rather have four real things than twelve maybes.

Compare the two versions side by side. The second should be dramatically shorter and every surviving item should trace back to a project. If it is still bloated, name the specific offending item and ask why it made the cut — that conversation is you calibrating Ghost's editorial judgment, which is exactly the skill Class 2 is training.

Crystallize — save the workflow

Once the tightened briefing feels right, freeze the behavior so you never re-teach it. Save it to Ghost as a reusable workflow with the relevance rules included, not just the format.

```
# SAVE THE REUSABLE WORKFLOW
```

Save this as a workflow called "daily-briefing".

It should always: pull from the current web, apply my saved priorities AND the relevance rules we just tuned (affects-a-project-or-cut, drop-empty-sections, rank-by-importance), and regenerate my briefing page in place.

From now on, when I say "today's briefing," run this workflow.

WHERE THIS GOES WRONG (AND HOW TO STEER)

- **It stays noisy after one nudge.** Don't vent — diagnose. Point at one specific item: "Why is this in here? It doesn't touch any of my projects." Making Ghost justify a cut recalibrates the whole filter.
- **It over-corrects to near-empty.** If everything got dropped, your threshold is now too high or your projects are too narrowly worded. Loosen slightly: "an item counts if it plausibly affects a project, not only if it definitely does."
- **It edits the words, not the selection.** If the rewrite is shorter sentences but the same items, restate that this is about *what to include*, not how to phrase it. Relevance ≠ brevity of prose.
- **New URL every run.** Remind it: same page, regenerate in place, today replaces yesterday. A briefing you can't bookmark isn't an artifact.
- **Priorities didn't stick.** If the briefing ignores your projects, memory priming failed. Re-run Step 1 and confirm recall before rebuilding.

Deliverable

WHAT YOU HAND IN

A live briefing page and evidence that better instructions cut the noise.

- ☐ A working, bookmarkable **URL** showing today's briefing in your locked format (one screen; headline + ≤ 3 bullets per item).
- ☐ A **before/after**: the noisy first version next to the tightened one, showing items dropped because they didn't affect a project.
- ☐ At least one **section that was dropped** entirely because nothing qualified — proof the threshold is real, not cosmetic.
- ☐ A saved **"daily-briefing" workflow** that regenerates the same page on command.

SCHEDULING — READ THIS, PROCTORS AND STUDENTS

Whether Ghost can wake itself on a timer is **uncertain and may vary**. This lab is written to work either way, because the lesson does not depend on it:

- **If Ghost can self-wake**: add one line to the workflow — *"produce this every morning without me asking."* The briefing page now updates on its own and you have a true unattended agent.
- **If it cannot**: the briefing is student-triggered — you say *"today's briefing"* and it regenerates. Still a persistent, re-generating artifact.

Either way you practiced the same core skill: **relevance filtering** plus a **persistent artifact that regenerates in place**. Don't let a scheduling question block the lab — build the triggered version first; add the timer only if Ghost supports it.

Stretch goals

- **"What changed since yesterday."** Have Ghost open each briefing with a one-line diff versus the last run — new developments only, so a returning reader sees the delta first.
- **Reader-set item cap**. Let the page accept a max item count ("show me the top 5") and make Ghost's ranking decide what survives the cut — a direct test of whether its importance ordering is any good.
- **Source links per item**. Add a link on each headline back to the primary source, so the briefing is auditable and you can trust its judgment enough to skip the click.
- **Two audiences, one workflow**. Save a second variant ("work briefing" vs "side-project briefing") that reuses the relevance engine with a different priority set.

FACILITATOR NOTES

- **The whole lab lives or dies on Step 3.** Budget the most time there. Students want to fix the writing; keep redirecting them to fix the *selection*. If they never see the noisy version, they miss the lesson — resist the urge to over-prime.
- **Push for specific projects up front.** "Tech and sports" cannot be relevance-filtered. Spend five minutes in Step 1 making stakes concrete; everything downstream gets easier.
- **Expect the over-correction.** Many will jump from firehose to nearly-empty. That's healthy — it means the threshold is now a live dial they're learning to turn. Coach them to loosen, not to abandon the rule.
- **Keep it in Discord.** All interaction is in the student's Discord channel. Ghost has its own cloud workspace and deploys the live page there; it cannot touch local files, so there is nothing to upload or install.
- **Scheduling is optional and unresolved.** Do not gate completion on self-wake. If a student's Ghost can't schedule, the triggered version is a full pass. Note which behavior their instance showed for later labs.
- **Common miss:** a new URL per run. Check that the deliverable is a stable bookmarkable link that regenerates in place — this is the "persistent artifact" half of the objective.